GPU-Accelerated Harris Corner Detection for Video Streams

Sami Enes Yilmaz* and Kadir Emre Avci†
*Computer Engineering Department, Hacettepe University, Ankara, Türkiye
†Graduate School of Science and Engineering, Hacettepe University, Ankara, Türkiye
E-mails: yilmaz.sami@hacettepe.edu.tr, avci.kadir@hacettepe.edu.tr

***Abstract*—This report presents a CUDA-based implementation of the Harris corner detector for video processing. The manuscript summarizes the algorithmic background, related literature, project architecture, and CUDA-oriented processing pipeline in IEEE format. It also reports profiler observations from representative video workloads and explains the implementation trade-offs used for the demonstrated configuration.**

***Index Terms*—Harris corner detection, CUDA, GPU acceleration, video processing, feature detection.**

I. Introduction

Corner detection is a core operation in computer vision pipelines including tracking, matching, motion analysis, and structure-from-motion. The Harris detector remains widely used due to its balance of computational cost, interpretability, and repeatability under moderate geometric and photometric variation. For high-frame-rate video, CPU implementations may become a bottleneck, motivating GPU acceleration.

This project implements a complete Harris pipeline with CUDA kernels for grayscale conversion, gradient estimation, second-moment matrix construction, Gaussian smoothing, response computation, and corner extraction. The implementation is integrated with OpenCV-based video input/output and frame-level visualization.

A practical and generic application scenario is *real-time corner-enhanced video preprocessing for downstream vision modules*. In this setting, Harris corners are overlaid or exported as lightweight geometric cues before optional tracking, motion analysis, or scene understanding stages.

Performance behavior is discussed using the collected Nsight Systems and Nsight Compute profiler evidence, with the demonstrated runs using the 5×5 filtering configuration supported by the codebase.

II. Related Work (Literature Review)

A. Foundations of Corner Detection

Harris and Stephens introduced an auto-correlation-based criterion that identifies points with significant intensity changes in two orthogonal directions [1]. Their response measure, based on local gradient covariance, is computationally simple and has become a foundational building block for keypoint analysis.

Shi and Tomasi later proposed selecting features according to the minimum eigenvalue of the second-moment matrix [2], improving practical tracking stability for many scenes. In practice, Harris and Shi–Tomasi are often treated as neighboring methods on the same formulation family.

B. Scale and Invariance-Oriented Detectors

SIFT introduced robust scale-space keypoint detection and high-dimensional descriptors with strong matching performance under scale and rotation transformations [3]. SURF then targeted lower computational cost through approximations based on integral images [4].

These methods generally improve invariance and matching quality, but they can be heavier than Harris when the objective is low-latency detection in fixed or mildly varying imaging conditions.

C. Real-Time Feature Alternatives

ORB combines oriented FAST detection with rotated BRIEF descriptors and is widely used in real-time visual SLAM and embedded vision contexts [5]. Compared with Harris-based designs, ORB can provide strong runtime efficiency while trading off some sensitivity characteristics and descriptor assumptions.

D. GPU Vision Context

GPU execution is well-suited to this pipeline because its core stages (gradient computation, smoothing, and response evaluation) are data-parallel at the pixel level. Since each output depends on local neighborhoods with limited inter-pixel dependency, Harris is a natural candidate for CUDA optimization in streaming video workloads.

In summary, prior methods motivate different trade-offs between invariance, descriptor richness, and runtime. For this project, Harris is selected as a practical detector for frame-wise CUDA acceleration where low-latency corner extraction is prioritized over heavy descriptor computation.

E. Literature Comparison with This Project

Table I summarizes representative differences between foundational studies and the current project implementation.

The comparison in Table I highlights that the selected project direction emphasizes efficient detector-stage acceleration rather than descriptor-heavy matching. This positioning is aligned with the practical objective of frame-wise corner extraction on video streams, where deterministic execution

TABLE I
LITERATURE COMPARISON AND POSITIONING OF THE CURRENT CUDA HARRIS PROJECT

Study / Method	Primary focus	Typical trade-off	Relation to this project
Harris–Stephens [1]	Corner response from second-moment structure tensor.	Good efficiency and interpretability; limited scale invariance.	Core mathematical basis used in the CUDA frame pipeline.
Shi–Tomasi [2]	Tracking-oriented corner quality criterion (minimum eigenvalue).	Often stable for tracking; still classical local-feature assumptions.	Alternative corner scoring reference; could be used as an ablation baseline.
SIFT [3]	Scale-invariant keypoint detection + high-dimensional descriptors.	Strong robustness, but heavier computation and descriptor cost.	Included as a robustness-oriented reference; not selected for low-latency target.
SURF [4]	Faster approximation to SIFT using integral-image ideas.	Better speed than SIFT; still descriptor-heavy relative to Harris-only detection.	Useful comparison for speed/robustness balance, but not as lightweight as Harris response-only pipeline.
ORB [5]	Real-time oriented detector+binary descriptor.	Very fast matching pipeline; different detector/descriptor design goal.	Runtime-efficient comparison context; current project focuses on CUDA Harris response stages.
This project (CUDA Harris)	GPU-accelerated frame-wise Harris corner extraction for video.	Prioritizes detector throughput and modular kernel decomposition over descriptor invariance.	Targets resolution-dependent real-time behavior and straightforward integration into preprocessing workflows.

flow and kernel modularity are prioritized. Based on this context, the following section details how the repository and CUDA modules are organized to support this implementation strategy.

III. SYSTEM AND PROJECT STRUCTURE

A. Repository-Level Organization

The repository is structured around kernel-by-kernel decomposition, with interface headers in `include/` and CUDA implementation files in `src/`. The entry point coordinates video decoding, per-frame GPU processing, and output encoding. This separation improves maintainability and enables independent profiling of each computational stage.

B. Execution Pipeline

The video stream is first decoded into an ordered sequence of individual frames using host-side video I/O. Each decoded frame is treated as a standalone 2D image for GPU processing, while frame order is preserved to keep temporal consistency in the output video. After processing one frame, the corner-annotated result is returned to the output writer and the pipeline advances to the next frame.

Within each frame iteration, kernels are executed in a fixed dependency order and intermediate buffers are reused in device memory to reduce transfer overhead. Detailed per-kernel responsibilities are provided in the next subsection, while Figure 2 shows the stage-level flow.

C. Kernel Responsibilities

The following kernel-level descriptions define the functional responsibility of each CUDA module in the end-to-end Harris processing pipeline.

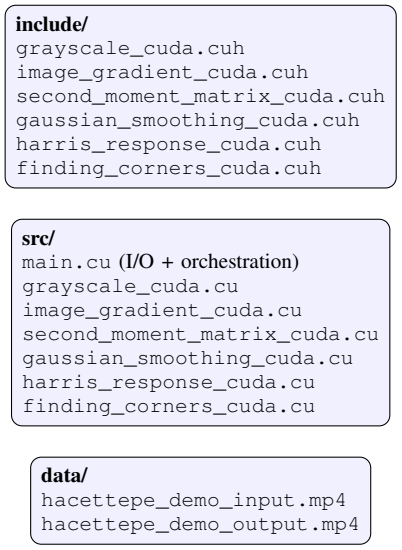

Fig. 1. Project/module structure used in the CUDA Harris pipeline implementation.

1) `grayscale_cuda.cu`: Converts input color frames to single-channel intensity values used by all downstream operations. This stage standardizes pixel representation, reduces memory bandwidth for subsequent kernels, and ensures derivative operators work on a consistent luminance domain rather than three independent color channels.

2) `image_gradient_cuda.cu`: Computes horizontal and vertical image derivatives (I_x and I_y), which form the basis of local structure analysis. In this project, Sobel filters are used for derivative estimation in both directions. The kernel maps one thread per pixel and applies local finite-difference style operations, producing gradient tensors that are consumed

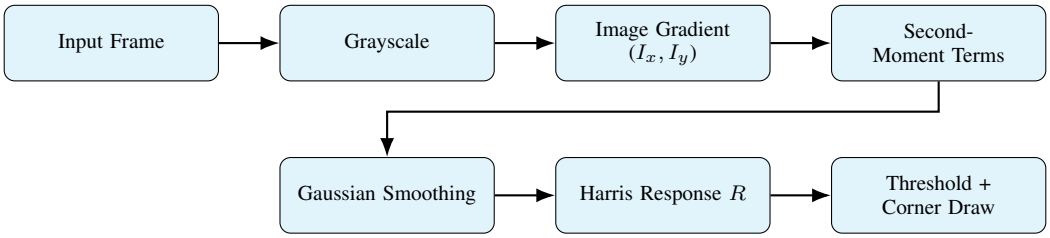

Fig. 2. Frame-wise CUDA processing flow from input frame to corner-annotated output (two-row stage layout).

directly by second-moment computation.

3) `second_moment_matrix_cuda.cu`*:* Builds per-pixel second-moment terms (I_x^2, I_y^2, and $I_x I_y$) before spatial smoothing. These quantities represent local directional energy and correlation, and they are stored in dedicated buffers to preserve precision through the subsequent smoothing stage.

4) `gaussian_smoothing_cuda.cu`*:* Applies Gaussian filtering to second-moment terms to stabilize neighborhood statistics and reduce noise sensitivity. By aggregating local evidence, this stage suppresses spurious gradient fluctuations and improves the numerical stability of the later Harris response evaluation.

5) `harris_response_cuda.cu`*:* Evaluates the Harris response function $R = \det(M) - k(\mathrm{trace}(M))^2$ for each pixel. The kernel combines smoothed tensor components into a scalar corner-strength map, where strong positive responses indicate candidate corner structures.

6) `finding_corners_cuda.cu`*:* Applies thresholding and corner-selection logic, then prepares output corner annotations. This stage filters weak responses, keeps salient locations, and writes overlay-ready coordinates/intensity cues for frame visualization in the final output stream.

IV. Method Details

A. Mathematical Formulation

Let an input RGB frame be denoted by channels $R(x, y)$, $G(x, y)$, and $B(x, y)$. The grayscale conversion used before derivative computation is

$$I(x, y) = 0.299\, R(x, y) + 0.587\, G(x, y) + 0.114\, B(x, y). \quad (1)$$

For filtering-oriented stages, padding is applied on the CPU side (OpenCV `copyMakeBorder`) to create an extended image $\tilde{I}(x, y)$ so neighborhood reads remain valid at boundaries. The selected padding type is replicate border (`cv::BORDER_REPLICATE`), which duplicates edge pixels into the halo region before data transfer to the device. Figure 3 shows a representative detection frame where multiple candidate corners appear directly on or very near the image borders (especially the bottom and lateral boundaries). This visual evidence motivates border padding: without padding, neighborhood filters at those boundary pixels would either be undefined or evaluated on truncated windows, often shifting maxima inward or discarding true border corners entirely. This choice is preferred over applying no padding because valid convolutions without padding would shrink the effective output domain (or force edge-pixel discard), reducing corner coverage near frame borders and making stage outputs less directly aligned with the original frame size. Because this replicated halo is materialized on the host for each matrix/image input, transfers are performed per padded buffer and not as a single monolithic `cudaMemcpy` that would otherwise assume one already-contiguous source layout for all operands. Using this padded representation, derivative and smoothing operations follow standard local convolution forms:

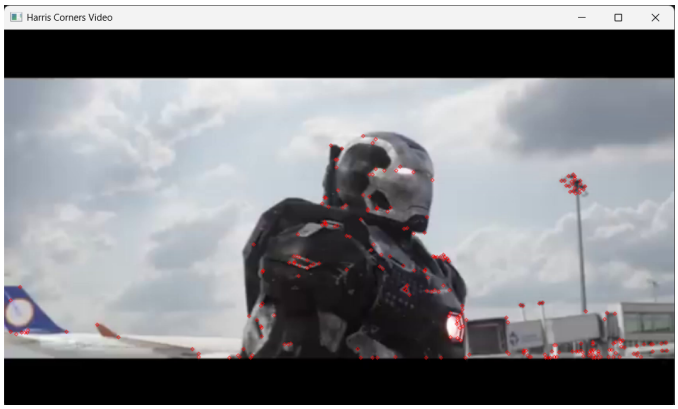

Fig. 3. Detected Harris corners near image edges.

$$I_x = K_x * \tilde{I}, \qquad I_y = K_y * \tilde{I}, \quad (2)$$

where K_x and K_y are Sobel derivative kernels (applied in horizontal and vertical directions, respectively) and $*$ denotes convolution. The implementation supports both 3×3 and 5×5 Sobel and Gaussian masks through the shared filter-size parameter; the current project demonstration and profiling analysis use the 5×5 configuration so the effect of neighborhood reuse is more visible.

The second-moment matrix is then computed as

$$M = \begin{bmatrix} G_\sigma * I_x^2 & G_\sigma * I_x I_y \\ G_\sigma * I_x I_y & G_\sigma * I_y^2 \end{bmatrix}, \quad (3)$$

where G_σ is Gaussian smoothing. The Harris response is

$$R = \det(M) - k\,(\mathrm{trace}(M))^2. \quad (4)$$

Pixels with large positive R are interpreted as corners.

B. Implementation Notes

The staged decomposition in this project favors debuggability and selective optimization. First, kernels can be validated independently against CPU-side references. Second, performance counters can be captured per stage to identify bottlenecks (e.g., memory bandwidth pressure in smoothing versus arithmetic intensity in response calculation). Third, parameter sweeps can be localized to one stage (e.g., Gaussian window size) without destabilizing the full pipeline.

The demonstrated 5×5 configuration increases neighborhood work from 9 to 25 samples per pixel compared with the smaller supported option, which makes convolution cost and data reuse easier to observe in profiling. The trade-off is stronger local smoothing and potentially better noise suppression at the cost of additional blur, more expensive convolutions, higher shared/global memory traffic, and larger boundary halos.

Stream concurrency is used selectively. In this implementation, streams are assigned to kernels that process pixels independently by chunks (e.g., grayscale conversion, second-moment element-wise products, and Harris response evaluation), where asynchronous host–device copies and compute can overlap effectively. Filtering kernels are excluded from this stream-based chunking path because they require neighborhood context and padded-border management, which makes chunk partitioning and overlap handling more complex than point-wise kernels.

Kernel launch configuration follows a 2D mapping strategy: both thread blocks and grids are defined in 2D so that kernel indexing directly matches image/matrix width-height dimensions, simplifying per-pixel coordinate mapping and coverage reasoning.

Profiling tooling is defined at a high level in implementation notes: `nsight-compute` is used for kernel-level metrics, while `nsight-systems` is used for timeline and overlap analysis.

The detector parameters used for the demonstration are now fixed in the report. The Harris constant is $k = 0.04$, the active Sobel/Gaussian filter size is 5×5, and corners are selected with a response threshold equal to 1% of the maximum Harris response in the current frame followed by local non-maximum suppression. The threshold is therefore frame-adaptive rather than a fixed absolute value.

The memory layout is also finalized. Reusable pinned host buffers are allocated once for the main per-frame tensors (I_x, I_y, I_{xx}, I_{yy}, I_{xy}, smoothed second-moment terms, Harris responses, and corner coordinates). Several CUDA stages then allocate temporary device buffers inside their stage functions, process the current frame or stream chunk, copy results back to the reusable host buffers, and free the temporary device storage. This staged allocation strategy is simple and transparent for profiling, while persistent device buffers remain outside the scope of the reported implementation.

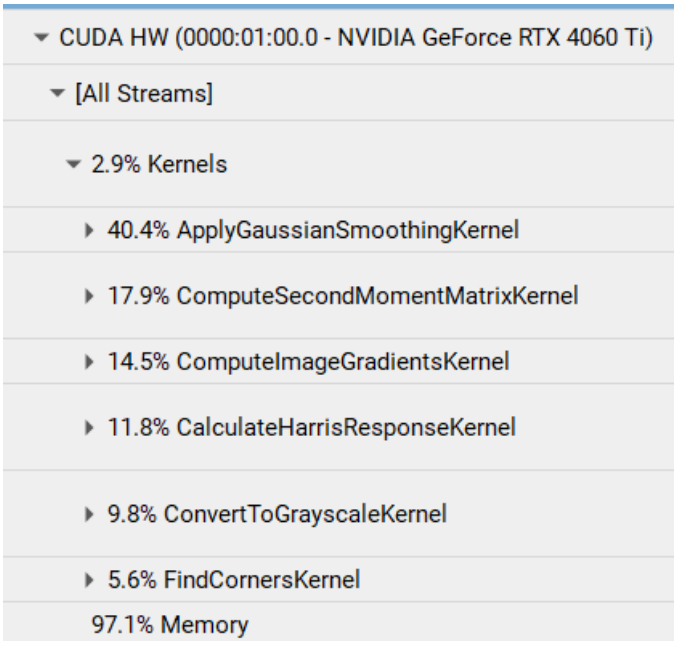

Fig. 4. Nsight Systems kernel/transfer distribution for a representative frame.

V. Experimental Results

This section reports the Nsight Systems and Nsight Compute observations collected from the CUDA Harris pipeline. The analysis focuses on four views: (i) kernel execution and stream distribution, (ii) full-frame execution structure, (iii) stream-enabled non-filtering stages, and (iv) a targeted shared-memory comparison for the Gaussian filtering kernel.

A. Kernel Execution and Stream Distribution

Figure 4 summarizes execution-time distribution for one representative frame. In this view, the default stream accounts for approximately **51.6%** of the GPU activity, while four auxiliary streams (#13–#16) each contribute nearly equal shares (**12.0–12.2%** each), giving an aggregate non-default share of about **48.4%**. This indicates that non-default stream work is relatively well balanced after task partitioning.

The observed distribution is consistent with the execution model. A higher default-stream share is expected because several stages remain serial due to stage dependencies, and filtering-related operations are intentionally not fully chunked across streams. The near-equal non-default stream shares are also consistent with the stream-enabled non-filtering path, where chunk sizes are selected to promote balanced overlap.

Within kernel execution, the `ApplyingGaussianSmoothing` kernel occupies approximately **40%** of total kernel time and represents the dominant compute hotspot in the profile. This is expected because Gaussian smoothing is a neighborhood-based convolution over full-frame second-moment buffers, leading to higher per-pixel memory traffic and arithmetic work than element-wise stages such as grayscale conversion, tensor product formation, or Harris response evaluation.

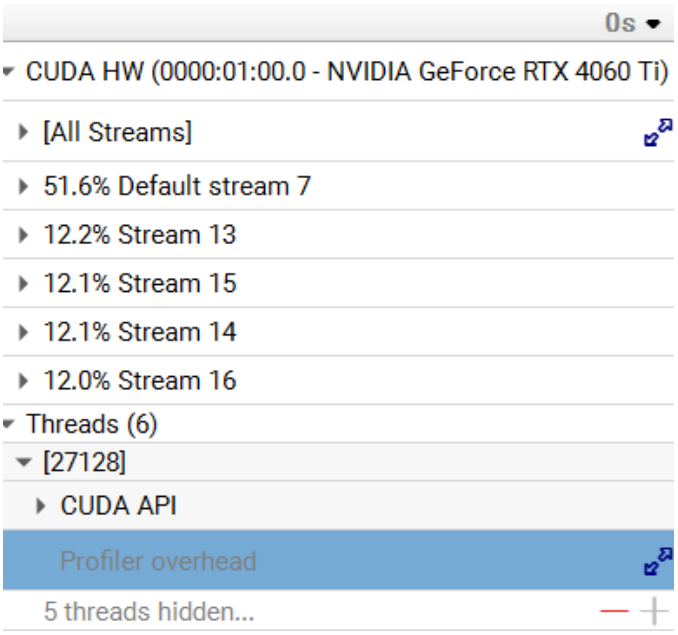

Fig. 5. Nsight Systems stream-level distribution across the default and auxiliary streams.

The timeline is memory-transfer heavy relative to pure compute, which is consistent with the current design. Transfer activity is amplified by host-side padding and staging operations performed before filtering kernels are launched. In this implementation, padding is created on the CPU using replicated borders, and padded matrices are subsequently copied to the device. Therefore, the measured transfer share reflects both unavoidable host-device communication and CPU-prepared padded-buffer workflow decisions, not only kernel inefficiency.

B. Full-Frame Timeline Interpretation

Figure 6 presents a full-frame timeline view from Nsight Systems. The shown interval corresponds to one frame among many in the input video stream; the same stage sequence is repeated for each frame until the stream ends.

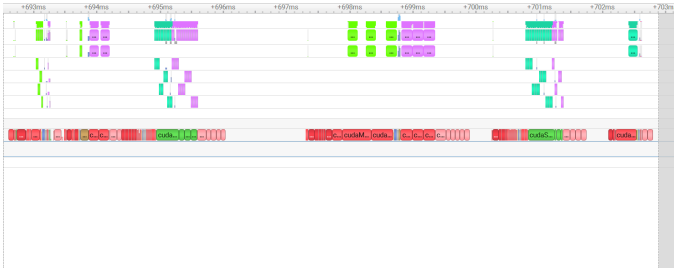

Fig. 6. Full-frame Nsight Systems timeline view (one frame instance within the full video).

The CUDA API activity is visible in the lower part of the timeline (last row region in the profiler layout), where asynchronous copies, synchronizations, and launch calls appear as host-side API events. Above these API rows, GPU execution lanes show actual device work. Color changes in the timeline correspond to different categories of CUDA operations (e.g., memory operations versus kernel launches), helping identify where control, transfer, and compute phases alternate during a frame.

Even in this frame-level snapshot, overlap opportunities can be observed: some stages are initiated with stream-based asynchronous CUDA calls, allowing command submission and device execution to proceed without strict global serialization. This behavior becomes clearer in the next subsection.

C. Stream Concurrency in Non-Filtering Stages

Figure 7 details the stream-enabled part of the pipeline for non-filtering stages, highlighting one of the main strengths of this project.

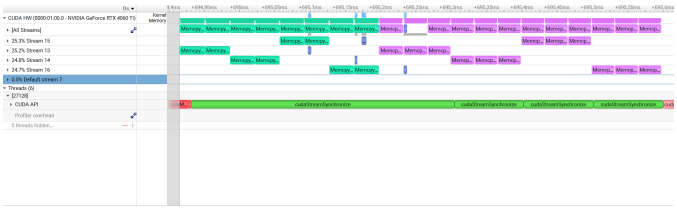

Fig. 7. Detailed Nsight Systems view of stream usage in a non-filtering stage.

To expose GPU-side parallelism clearly in Nsight Systems, memory copies and kernel launches were reorganized from a single loop into three distinct loops: one loop for batched/partitioned `cudaMemcpyAsync` submissions, one loop for kernel launches, and one loop for post-launch synchronization/collection. This restructuring reduces host-side interleaving that can hide overlap patterns in the timeline and makes concurrent execution across streams easier to observe.

The resulting trace shows that independent chunks in non-filtering stages can progress concurrently: while one chunk is executing on device, another chunk transfer can already be in flight. This improves pipeline utilization by overlapping communication and computation. As expected, filtering stages were not moved to the same chunked-stream strategy because neighborhood dependencies and padding-border handling make safe partitioning more constrained than point-wise or element-wise stages.

Overall, Nsight Systems confirms that stream-based asynchronous design is effective for independent non-filtering operations, while transfer-heavy behavior is primarily explained by CPU-side padded-buffer preparation required by filtering stages.

D. Shared-Memory Analysis of Gaussian Filtering

Because the Gaussian smoothing stage is the dominant kernel-level hotspot, a focused Nsight Compute comparison was performed for `ApplyGaussianSmoothingKernel`. The experiment used only the first frame of the 1080p input video so that both variants were evaluated on the same image content and frame resolution. A 5×5 Gaussian filter was selected for demonstration and analysis to amplify the effect

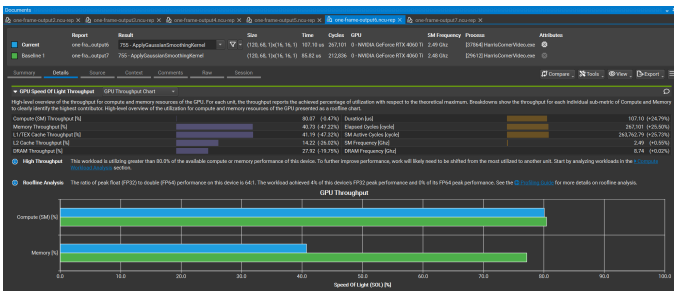

Fig. 8. Nsight Compute comparison of `ApplyGaussianSmoothingKernel` on the first 1080p frame using a 5×5 Gaussian filter. The blue current run (`one-frame-output6`) is the non-shared-memory implementation, and the green baseline run (`one-frame-output7`) is the shared-memory implementation.

of neighborhood reuse: each output pixel samples 25 input values, making redundant global-memory reads more visible than in the smaller-filter case.

Figure 8 compares `one-frame-output6`, which does not use shared memory, against `one-frame-output7`, which uses shared-memory tiling. In the Nsight Compute comparison view, `one-frame-output6` is the blue current run and `one-frame-output7` is the green baseline run. This distinction is important because the green shared-memory bars show both higher memory throughput and lower execution time.

The measured duration confirms that shared memory improves the 5×5 Gaussian kernel for this input. The non-shared-memory run requires **107.10** μ**s**, whereas the shared-memory run requires only **85.82** μ**s**. Thus, the non-shared-memory implementation is approximately **24.8%** slower than the shared-memory implementation, and shared-memory tiling reduces the measured duration by about **19.9%** relative to the non-shared-memory run.

The memory-throughput result should be read in the same direction. The blue non-shared-memory run reports **40.73%** memory throughput, while the green shared-memory baseline is higher (approximately **77%**, based on the Nsight Compute delta shown in the comparison). Therefore, the shared-memory version is not merely reducing memory activity; it is using the memory hierarchy more effectively while finishing sooner. This is consistent with the intended optimization: staging a block tile and halo in shared memory increases local data reuse for neighboring threads, so the kernel can sustain higher useful memory throughput during the convolution.

Compute throughput stays close to **80%** for both runs, so the key difference is the interaction between memory access and computation rather than a large change in arithmetic utilization. For the first 1080p frame with a 5×5 filter and 16×16 CUDA blocks, shared-memory tiling provides a clear performance benefit for `ApplyGaussianSmoothingKernel`. No additional failure modes were identified from the demonstration outputs beyond the expected Harris-detector behavior: textured regions and strong borders can produce dense corner clusters when the adaptive threshold is permissive.

VI. Conclusion

This project implemented a complete GPU-accelerated Harris corner detection pipeline for video streams. Each frame is converted to grayscale, processed with Sobel gradient filters, transformed into second-moment matrix components, smoothed with a Gaussian filter, evaluated with the Harris response equation, and post-processed with thresholding and local non-maximum suppression before detected corners are drawn back onto the output video. The implementation is organized as separate CUDA stages so that each part of the detector can be tested, profiled, and optimized independently while remaining integrated with OpenCV video input/output.

The implementation supports the main properties required for the demonstration videos. It can process the included 480p, 720p, and 1080p inputs, uses four CUDA streams for chunked asynchronous execution in independent non-filtering stages, and supports both 3×3 and 5×5 Sobel/Gaussian filter sizes through the common filter-size parameter. The current demonstration and the profiling discussion use the 5×5 configuration, not because the project is limited to that size, but because the larger neighborhood makes the cost of convolution and the benefit of data reuse easier to analyze. Detector parameters are fixed for the reported runs: the Harris constant is $k = 0.04$, and corner extraction uses a frame-adaptive threshold equal to 1% of the maximum Harris response followed by local non-maximum suppression.

CPU-side replicate padding is used before the filtering kernels so that Sobel and Gaussian neighborhoods remain valid even at image boundaries. This choice keeps output dimensions aligned with the original frame and avoids discarding potential border corners, which is important for video visualization where detections near frame edges should still be represented. The trade-off is additional host-side preparation and padded-buffer transfer cost, but the current design is justified for correctness, simpler boundary handling, and clear stage-by-stage profiling.

The profiling results show that the pipeline benefits from stream-based overlap in independent stages, with non-default streams carrying a balanced share of the work while serial dependencies and filtering remain on the default-stream path. Nsight Systems also shows that the execution is transfer-heavy, largely because of host-prepared padded buffers for filtering. At the kernel level, Gaussian smoothing is the dominant hotspot, occupying about 40% of total kernel time in the representative profile. The focused Nsight Compute result for the first 1080p frame shows that shared-memory tiling improves the demonstrated 5×5 Gaussian kernel from 107.10 μs to 85.82 μs, so shared memory is a useful local optimization, while the broader project outcome is the complete CUDA Harris video pipeline with configurable resolution and filter-size support.

References

[1] C. Harris and M. Stephens, "A combined corner and edge detector," in *Proceedings of the Alvey Vision Conference*, 1988, pp. 147–151.

[2] J. Shi and C. Tomasi, "Good features to track," in *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1994, pp. 593–600.
[3] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
[4] H. Bay, A. Ess, T. Tuytelaars, and L. V. Gool, "Speeded-up robust features (surf)," *Computer Vision and Image Understanding*, vol. 110, no. 3, pp. 346–359, 2008.
[5] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "Orb: An efficient alternative to sift or surf," in *Proceedings of IEEE International Conference on Computer Vision (ICCV)*, 2011, pp. 2564–2571.